

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
VIỆN TRÍ TUỆ NHÂN TẠO**



**BÁO CÁO MÔN HỌC
XỬ LÝ NGÔN NGỮ TỰ NHIÊN VÀ DỮ LIỆU LỚN**

**ĐỀ TÀI:
TÌM HIỂU VÀ TRIỂN KHAI KIẾN TRÚC
HIERARCHICAL TRANSFORMER CHO BÀI
TOÁN SỬA LỖI CHÍNH TẢ TIẾNG VIỆT**

Nhóm sinh viên thực hiện:

1. Nguyễn Ngọc Bình An - 24022247
2. Nguyễn Đình Phú - 24022427
3. Nguyễn Đoàn Nhật Minh - 24022403
4. Phạm Lê Việt Đức - 24022296

Giảng viên hướng dẫn: TS. Trần Hồng Việt

MỞ ĐẦU

Sửa lỗi chính tả tự động là một trong những bài toán nền tảng và cốt lõi nhất của lĩnh vực Xử lý Ngôn ngữ Tự nhiên. Trong kỷ nguyên số hóa, sự bùng nổ của thông tin trên mạng xã hội, các cổng thông tin điện tử, hệ thống chăm sóc khách hàng tự động (Chatbot), cũng như các hệ thống nhận dạng tiếng nói (ASR) và nhận dạng chữ viết quang học (OCR) đã tạo ra một khối lượng văn bản khổng lồ nhưng chứa đựng nhiều sai lệch về chính tả. Văn bản sai chính tả không chỉ làm suy giảm trải nghiệm người dùng mà còn gây ảnh hưởng nghiêm trọng đến hiệu năng của các tác vụ NLP hạ nguồn như dịch máy, phân tích cảm xúc, trích xuất thông tin và tìm kiếm thông tin.

Khác với tiếng Anh và các ngôn ngữ hệ chữ Latinh không thanh điệu, tiếng Việt mang những đặc thù ngôn ngữ học rất riêng biệt: là ngôn ngữ đơn lập, đơn âm tiết, giàu thanh điệu (với 6 thanh điệu khác nhau) và sử dụng hệ thống dấu phụ phong phú. Người dùng tiếng Việt nhập liệu văn bản chủ yếu thông qua các bộ gõ như Telex hoặc VNI. Do đó, các lỗi chính tả tiếng Việt phát sinh từ nhiều nguồn khác nhau: từ việc gõ phím kè, gõ thừa/thiếu phím chuyển đổi dấu, nhầm lẫn thanh điệu do phát âm vùng miền (đặc biệt là nhầm lẫn giữa dấu hỏi và dấu ngã), nhầm lẫn phụ âm đầu/cuối, cho đến việc sử dụng ngôn ngữ mạng (teencode) và các từ viết tắt phổ biến.

Dự án này tập trung nghiên cứu và cài đặt kiến trúc **Hierarchical Transformer Encoder** phục vụ bài toán phát hiện và sửa lỗi chính tả tiếng Việt. Với cấu hình chỉ với khoảng 1 triệu tham số, mô hình giải quyết triệt để bài toán tài nguyên, có thể huấn luyện mượt mà trên các GPU phổ thông đồng thời duy trì độ chính xác phát hiện lỗi và phục hồi chính tả ở mức khá. Hệ thống được đóng gói hoàn chỉnh từ pipeline sinh dữ liệu tổng hợp 4,66 triệu mẫu, pipeline huấn luyện tối ưu hóa bộ nhớ theo luồng (streaming JSONL), cho đến ứng dụng web tương tác toàn diện được phát triển bằng FastAPI và React.

Mục lục

MỞ ĐẦU	i
1 TỔNG QUAN VỀ ĐỀ TÀI	1
1.1 Giới thiệu bài toán và tầm quan trọng	1
1.1.1 Bài toán sửa lỗi chính tả tiếng Việt	1
1.1.2 Tại sao việc sửa lỗi chính tả tiếng Việt lại quan trọng?	1
1.2 Đặc thù ngôn ngữ và các dạng lỗi chính tả trong tiếng Việt	2
1.2.1 Lỗi gõ phím	2
1.2.2 Lỗi ngữ âm và dấu thanh vùng miền	2
1.2.3 Lỗi viết tắt và ngôn ngữ mạng	2
1.3 Mục tiêu cụ thể của dự án	2
1.4 Phạm vi của dự án	3
2 BỘ DỮ LIỆU VÀ QUY TRÌNH TẠO DỮ LIỆU NHIỀU TỔNG HỢP	4
2.1 Nguồn dữ liệu sạch	4
2.2 Kiểm soát chống rò rỉ dữ liệu	5
2.3 Quy trình sinh nhiều tổng hợp	5
2.4 Không gian từ vựng và phân chia dữ liệu	5
2.4.1 Xây dựng từ vựng	5
2.4.2 Phân chia Train / Validation	5
3 KIẾN TRÚC MÔ HÌNH HIERARCHICAL TRANSFORMER ENCODER	7
3.1 Kiến trúc mô hình	7
3.1.1 Bộ mã hóa cấp ký tự và cơ chế Masked Mean Pooling	7
3.1.2 Tầng kết hợp đặc trưng (Feature Fusion Layer)	8
3.1.3 Bộ mã hóa cấp từ với chia sẻ trọng số	8
3.1.4 Đầu ra đa nhiệm và ràng buộc trọng số (Weight Tying)	9
3.1.5 Hàm mất mát tổng hợp	9
3.1.6 Bảng phân rã chi tiết tham số cấu hình	10
4 HUẤN LUYỆN, THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ	11
4.1 Môi trường thực nghiệm và kỹ thuật tối ưu hóa	11
4.1.1 Hạ tầng phần cứng và môi trường phần mềm	11
4.1.2 Kỹ thuật huấn luyện tối ưu bộ nhớ	11
4.1.3 Thiết lập siêu tham số huấn luyện (Hyperparameters)	12
4.2 Đánh giá kết quả và hiệu năng mô hình	12
4.2.1 Mức độ hội tụ của hàm mất mát	12
4.2.2 Hiệu năng phát hiện lỗi	12
4.2.3 Hiệu năng sửa lỗi toàn diện (End-to-End Correction)	13
5 TRIỂN KHAI ỨNG DỤNG VÀ THỰC NGHIỆM WEB DEMO	14
5.1 Kiến trúc hệ thống Web tương tác	14
5.2 Các tính năng kỹ thuật nổi bật	15
6 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	16
6.1 Kết luận	16
6.2 Hạn chế của mô hình	16
6.3 Hướng phát triển trong tương lai	17
PHÂN CÔNG CÔNG VIỆC	18

Chương 1

TỔNG QUAN VỀ ĐỀ TÀI

1.1 Giới thiệu bài toán và tầm quan trọng

1.1.1 Bài toán sửa lỗi chính tả tiếng Việt

Bài toán sửa lỗi chính tả tiếng Việt nhằm mục đích chuyển đổi một chuỗi văn bản đầu vào $X = (x_1, x_2, \dots, x_N)$ có chứa các token sai chính tả thành chuỗi văn bản chuẩn $Y = (y_1, y_2, \dots, y_N)$ đúng chính tả, đúng ngữ pháp và bảo toàn trọn vẹn ngữ nghĩa của câu. Trong phạm vi nghiên cứu của đề tài, bài toán được mô hình hóa dưới dạng bài toán phân loại và sửa lỗi token căn chỉnh 1-1, trong đó với mỗi token x_i ở vị trí thứ i :

1. **Phát hiện lỗi:** Hệ thống xác định xem x_i có phải là token bị lỗi chính tả hay không thông qua nhãn nhị phân $\hat{d}_i \in \{0, 1\}$, với $\hat{d}_i = 0$ biểu thị token đúng và $\hat{d}_i = 1$ biểu thị token có lỗi chính tả.
2. **Sửa lỗi:** Nếu token được xác định là bị lỗi ($\hat{d}_i = 1$), hệ thống dự đoán từ đúng $\hat{y}_i \in \mathcal{V}_{\text{word}}$ trong không gian từ vựng chuẩn $\mathcal{V}_{\text{word}}$ để thay thế cho token x_i . Ngược lại, nếu $\hat{d}_i = 0$, từ gốc x_i được giữ nguyên.

1.1.2 Tại sao việc sửa lỗi chính tả tiếng Việt lại quan trọng?

Hệ thống sửa lỗi chính tả tiếng Việt tự động đóng vai trò nền tảng trong cả nghiên cứu và ứng dụng thực tiễn dựa trên ba trụ cột cốt lõi. Thứ nhất, việc phát hiện và sửa lỗi giúp nâng cao chất lượng cho các tác vụ NLP bằng cách đóng vai trò là module tiền xử lý và hậu xử lý thiết yếu cho hệ thống nhận dạng tiếng nói (ASR) và nhận dạng chữ quang học (OCR), từ đó giảm thiểu tỷ lệ từ ngoài từ vựng (OOV) để duy trì độ chuẩn xác cho các mô hình dịch máy, phân tích cảm xúc và trích xuất thông tin. Thứ hai, hệ thống cải thiện trực tiếp hiệu năng của các công cụ tìm kiếm thông qua cơ chế tự động hiệu đính lỗi trong câu truy vấn (Query Auto-Correction), giúp nhận diện đúng ý định của người dùng và tăng độ liên quan của kết quả tìm kiếm. Cuối cùng, công nghệ này tối ưu hóa trải nghiệm người dùng khi đóng vai trò như một trợ lý biên tập thông minh trong soạn thảo tài liệu số, hệ thống giáo dục trực tuyến và chatbot tự động, bảo đảm tính chuẩn mực của tiếng Việt trên môi trường số.

1.2 Đặc thù ngôn ngữ và các dạng lỗi chính tả trong tiếng Việt

Tiếng Việt có những nét đặc thù dẫn đến các dạng lỗi chính tả đa dạng và phức tạp hơn so với các ngôn ngữ không có thanh điệu:

1.2.1 Lỗi gõ phím

- **Lỗi gõ Telex:** Người dùng gõ dư hoặc sai vị trí phím dấu (ví dụ: *bão* → *baox*, *bộ* → *booj*, *đường* → *dduwowngj*).
- **Lỗi gõ VNI:** Sử dụng các phím số để bỏ dấu nhưng bị nhận diện thành ký tự số (ví dụ: *bão* → *bao4*, *bộ* → *bo65*).
- **Lỗi phím kề QWERTY:** Bấm nhầm các phím liền kề trên bàn phím vật lý (ví dụ: phím *d* nhầm với *s*, *e*, *r*, *f*, *c*, *x*).
- **Lỗi xóa và đảo ký tự:** Do thao tác gõ quá nhanh dẫn đến thiếu ký tự (*phòng* → *phng*) hoặc hoán vị hai ký tự kề nhau (*phòng* → *póhng*).

1.2.2 Lỗi ngữ âm và dấu thanh vùng miền

- **Lỗi mất dấu thanh / mất toàn bộ dấu:** Người dùng bỏ qua dấu thanh (*bão* → *bao*) hoặc mất toàn bộ ký tự dấu phụ (*đường* → *duong*).
- **Nhầm lẫn dấu hỏi và dấu ngã:** Hiện tượng cực kỳ phổ biến ở phương ngữ miền Trung và miền Nam do cách phát âm không phân biệt giữa âm vị hỏi và ngã (ví dụ: *bão* → *bảo*, *suy nghĩ* → *suy nghĩ*, *vẽ* → *vẻ*).
- **Nhầm lẫn phụ âm đầu:** Xuất phát từ phát âm tương đồng giữa các cặp phụ âm như *tr/ch*, *s/x*, *d/gi/r*, *n/l* (ví dụ: *trồng cây* → *chồng cây*, *giành độc lập* → *dành độc lập*).

1.2.3 Lỗi viết tắt và ngôn ngữ mạng

Trong giao tiếp thường nhật trên mạng xã hội và tin nhắn, người dùng thường có thói quen viết tắt các từ đơn phổ biến: *không* → *k*, *ko*, *kh*, *khg*; *được* → *đc*, *dc*; *chơi* → *chs*; *nữa* → *nx*; *gì* → *j*...

1.3 Mục tiêu cụ thể của dự án

- **Triển khai kiến trúc Hierarchical Transformer Encoder:** Xây dựng một mạng học sâu hai cấp (Character-level và Word-level) có khả năng kết hợp biểu diễn ký tự hình thái và ngữ cảnh câu hai chiều.

- **Tối ưu hóa tài nguyên phần cứng:** Thu gọn số lượng tham số xuống xấp xỉ 1 triệu tham số thông qua chia sẻ trọng số ALBERT và ràng buộc ma trận Weight Tying, cho phép huấn luyện và phục vụ hiệu quả trên các thiết bị tài nguyên thấp.
- **Xây dựng pipeline sinh dữ liệu quy mô lớn:** Thiết lập pipeline trích xuất từ 8,55 triệu câu Wikipedia tiếng Việt sạch, loại bỏ triệt để rò rỉ dữ liệu đối với tập kiểm thử Viwiki-Spelling và tổng hợp 10 dạng nhiễu thực tế.
- **Đóng gói sản phẩm hoàn chỉnh:** Phát triển ứng dụng Web Full-stack (FastAPI + React) phục vụ việc trải nghiệm trực quan theo thời gian thực.

1.4 Phạm vi của dự án

Dự án được xác định và giới hạn trong các phạm vi kỹ thuật, dữ liệu và triển khai cụ thể sau:

- **Giới hạn kích thước đầu vào:** Mô hình học sâu tiếp nhận các chuỗi câu có độ dài tối đa $T \leq 192$ token và mỗi token có độ dài tối đa $C \leq 32$ ký tự
- **Không gian từ vựng và biểu diễn:** Mô hình sử dụng không gian từ vựng cố định gồm 9.500 từ vựng và 400 ký tự. Kích thước này được tính toán tối ưu để cân bằng giữa độ bao phủ ngữ nghĩa và chi phí tính toán tham số

Chương 2

BỘ DỮ LIỆU VÀ QUY TRÌNH TẠO DỮ LIỆU NHIỀU TỔNG HỢP

2.1 Nguồn dữ liệu sạch

Để xây dựng bộ dữ liệu huấn luyện quy mô lớn, nhóm sử dụng bản chính thức của Wikipedia tiếng Việt. Tập nguồn `viwiki-latest-pages-articles.xml.bz2` có dung lượng nén khoảng 1,1 GB.

Quy trình trích xuất câu sạch được thực hiện như sau:

1. **Lọc không gian tên:** Chỉ xử lý các bài viết thuộc Article Namespace, loại bỏ hoàn toàn các trang chuyển hướng.
2. **Loại bỏ Markup và HTML:** Dọn dẹp các thẻ HTML, thẻ chú thích `<ref>`, bảng biểu wiki `{|...|}`, bản mẫu wiki lồng nhau `{{...}}`, liên kết trong `[[...]]`, và tiêu đề phân mục.
3. **Phân tách câu và chuẩn hóa Unicode:** Tách câu dựa trên ranh giới dấu câu (`.`, `!`, `?`), chuẩn hóa toàn bộ văn bản về dạng Unicode dạng sẵn (NFC).
4. **Bộ lọc chất lượng nghiêm ngặt:**
 - Độ dài câu: Từ 5 đến 192 token (tương đương 24 đến 900 ký tự).
 - Tỷ lệ chữ cái: $\geq 52\%$ tổng độ dài ký tự của câu.
 - Yếu tố tiếng Việt: Phải chứa các ký tự nguyên âm hoặc dấu thanh đặc trưng tiếng Việt.
 - Tỷ lệ ký tự đặc biệt: $< 1.5\%$ để loại bỏ các câu công thức toán học hoặc chuỗi ký tự rác.
 - Loại bỏ trùng lặp: Áp dụng mã băm SHA-256 trên chuỗi chữ thường để khử trùng lặp hoàn toàn.

Kết quả trích xuất thu được tổng dung lượng văn bản thô đạt 2 GB.

2.2 Kiểm soát chống rò rỉ dữ liệu

Để đảm bảo tính khách quan tuyệt đối khi đánh giá trên tập benchmark công khai **Viwiki-Spelling**, toàn bộ mã định danh bài viết (`page_id`) của các tài liệu trong Viwiki-Spelling (gồm 104 bài viết) được thu thập trước. Bất kỳ câu nào xuất phát từ các `page_id` này đều bị loại bỏ hoàn toàn khỏi quá trình trích xuất văn bản huấn luyện và kiểm định.

2.3 Quy trình sinh nhiễu tổng hợp

Từ tập câu sạch, nhóm tiến hành sinh dữ liệu lỗi căn chỉnh 1-1 với các tham số: tỷ lệ câu giữ sạch 15%, tỷ lệ token bị chèn lỗi 13%, tối đa 4 lỗi trên mỗi câu, và tạo 2 biến thể lỗi cho mỗi câu. Hệ thống áp dụng 10 loại lỗi chính tả để tạo nhiễu ngẫu nhiên với các trọng số:

Lỗi chính tả	Trọng số sinh	Mô tả
telex	0.18	Lỗi gõ phím Telex (<i>bão</i> → <i>baox</i> , <i>bộ</i> → <i>booj</i>)
drop_tone	0.12	Xóa dấu (<i>bão</i> → <i>bao</i> , <i>bộ</i> → <i>bô</i>)
keyboard	0.12	Bấm nhầm phím kề (<i>phòng</i> → <i>phongg</i>)
drop_diacritic	0.10	Xóa toàn bộ dấu phụ và thanh điệu (<i>đường</i> → <i>duong</i>)
swap_tone	0.10	Tráo đổi dấu hỏi và dấu ngã (<i>bão</i> → <i>bảo</i>)
abbreviation	0.10	Viết tắt / Teencode (<i>không</i> → <i>ko</i>)
vni	0.08	Lỗi gõ phím VNI (<i>bão</i> → <i>bao4</i> , <i>bộ</i> → <i>bo65</i>)
delete	0.08	Xóa ngẫu nhiên 1 ký tự trong từ (<i>ngôn</i> → <i>ngn</i>)
transpose	0.06	Tráo đổi 2 ký tự liền kề (<i>phòng</i> → <i>pôhng</i>)
consonant	0.06	Lẫn phụ âm đầu tương đồng (<i>tr/ch, s/x, d/gi, n/l</i>)

Bảng 2.1: Chi tiết 10 toán tử sinh lỗi tổng hợp trong tập dữ liệu.

2.4 Không gian từ vựng và phân chia dữ liệu

2.4.1 Xây dựng từ vựng

- **Word Vocabulary (`word_vocab_full.json`)**: Gồm 9.500 từ phổ biến nhất trong tập văn bản sạch, kèm theo các token đặc biệt `<pad>`, `<unk>`.
- **Character Vocabulary (`char_vocab_full.json`)**: Gồm 400 ký tự đơn lẻ phổ biến nhất (gồm bảng chữ cái tiếng Việt, dấu thanh, ký tự số và ký hiệu).

2.4.2 Phân chia Train / Validation

Việc phân chia được thực hiện dựa trên hàm băm SHA-256 của `page_id` với tỷ lệ 98% cho Train và 2% cho Validation. Phương pháp này đảm bảo không có sự trùng lặp dữ liệu giữa hai tập:

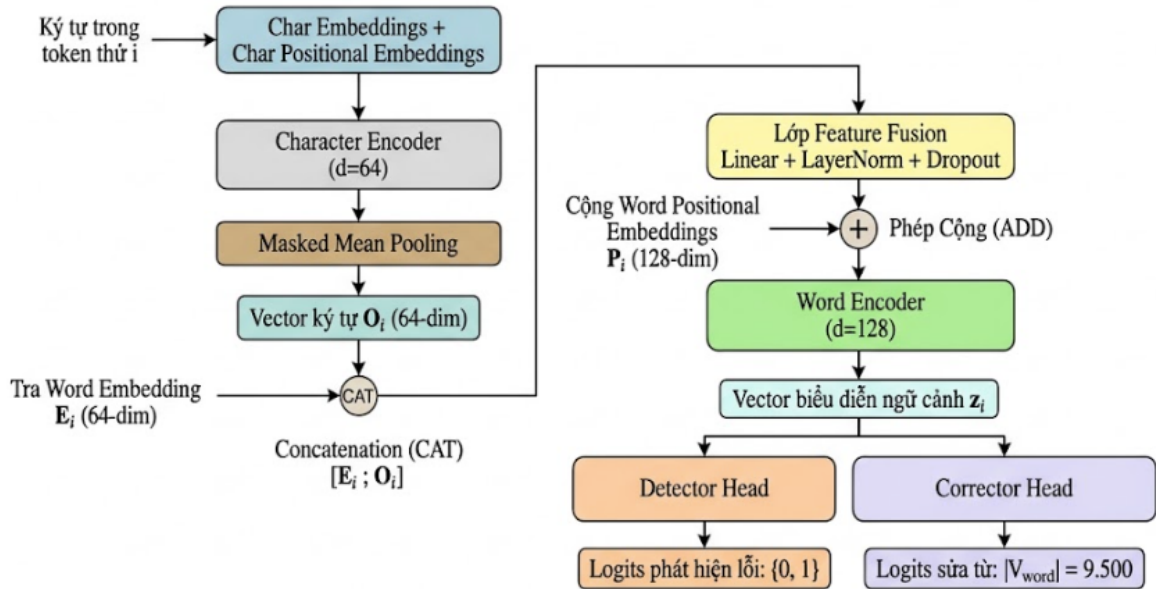
- **Tập Train (`train_full.jsonl`):** 4.564.618 mẫu (Dung lượng 6,04 GB).
- **Tập Validation (`validation_full.jsonl`):** 94.836 mẫu (Dung lượng 124 MB).
- **Tổng số mẫu:** 4.659.454 mẫu dữ liệu được kiểm thực căn chỉnh token 100% và được tải lên trên [Hugging Face](#).

Chương 3

KIẾN TRÚC MÔ HÌNH HIERARCHICAL TRANSFORMER ENCODER

3.1 Kiến trúc mô hình

Mô hình **HierarchicalSpellingCorrector** được xây dựng theo nguyên lý phân cấp hai tầng (Character-to-Word Hierarchy): Tầng ký tự (Character-level) trích xuất đặc trưng hình thái chữ viết cục bộ bên trong từng từ độc lập, và Tầng từ (Word-level) nắm bắt ngữ cảnh ngữ nghĩa đa chiều trên toàn bộ câu.



3.1.1 Bộ mã hóa cấp ký tự và cơ chế Masked Mean Pooling

Giả sử câu đầu vào có T token ($T \leq 192$), mỗi token x_i chứa tối đa C ký tự ($c_{i,1}, c_{i,2}, \dots, c_{i,C}$) ($C \leq 32$). Tensor ký tự đầu vào có kích thước (B, T, C) được reshape thành $(B \times T, C)$ để bảo đảm cơ chế Self-Attention cấp ký tự chỉ tính toán cục bộ bên trong từng token và không nhìn chéo giữa các token khác nhau.

Biểu diễn ban đầu của ký tự thứ j trong token i được tính bằng tổng của vector nhúng ký tự và vector nhúng vị trí ký tự:

$$\mathbf{h}_{i,j}^{(0)} = \mathbf{E}_{\text{char}}(c_{i,j}) + \mathbf{P}_{\text{char}}(j) \in \mathbb{R}^{d_{\text{char}}} \quad (3.1)$$

Chuỗi biểu diễn ký tự được xử lý qua $L_{\text{char}} = 2$ lớp Transformer Encoder với cấu trúc Pre-LayerNorm (`norm_first=True`), số đầu chú ý $H_{\text{char}} = 4$, kích thước ẩn $d_{\text{char}} = 64$ và kích thước Feed-Forward $d_{\text{ff}} = 256$:

$$\mathbf{H}_i = \text{TransformerEncoder}_{\text{char}}(\mathbf{h}_{i,1}^{(0)}, \dots, \mathbf{h}_{i,C}^{(0)}) \in \mathbb{R}^{C \times d_{\text{char}}} \quad (3.2)$$

Để chuyển đổi C vector ký tự đầu ra thành một vector duy nhất đại diện cho token x_i , mô hình áp dụng cơ chế **Masked Mean Pooling** (trung bình có mặt nạ):

$$\mathbf{O}_i = \frac{\sum_{j=1}^C m_{i,j} \cdot \mathbf{h}_{i,j}^{(L_{\text{char}})}}{\max(1, \sum_{j=1}^C m_{i,j})} \in \mathbb{R}^{d_{\text{char}}} \quad (3.3)$$

trong đó $m_{i,j} \in \{0, 1\}$ là mặt nạ chỉ định ký tự thực tế ($m_{i,j} = 1$ nếu ký tự hợp lệ và $m_{i,j} = 0$ nếu là vị trí đệm `<pad>`). Cơ chế này triệt tiêu hoàn toàn nhiễu từ các ký tự padding.

3.1.2 Tầng kết hợp đặc trưng (Feature Fusion Layer)

Token x_i được tra cứu đồng thời qua bảng từ vựng cấp từ để lấy vector nhúng từ $\mathbf{E}_{\text{word}}(x_i) \in \mathbb{R}^{d_{\text{word_emb}}}$ ($d_{\text{word_emb}} = 64$). Vector nhúng từ và vector ký tự được ghép nối (concatenation) thành vector 128 chiều, sau đó đưa qua tầng chiếu tuyến tính kết hợp chuẩn hóa LayerNorm và Dropout:

$$\begin{aligned} \mathbf{z}_i^{(0)} = & \text{Dropout}(\text{LayerNorm}(\mathbf{W}_{\text{fusion}}[\mathbf{E}_{\text{word}}(x_i); \mathbf{O}_i] + \mathbf{b}_{\text{fusion}})) \\ & + \mathbf{P}_{\text{word}}(i) \in \mathbb{R}^{d_{\text{word}}} \end{aligned} \quad (3.4)$$

trong đó $\mathbf{W}_{\text{fusion}} \in \mathbb{R}^{128 \times 128}$, $\mathbf{b}_{\text{fusion}} \in \mathbb{R}^{128}$, và $\mathbf{P}_{\text{word}}(i) \in \mathbb{R}^{128}$ là ma trận nhúng vị trí từ học được (192×128).

3.1.3 Bộ mã hóa cấp từ với chia sẻ trọng số

Chuỗi vector toàn câu $\mathbf{Z}^{(0)} = (\mathbf{z}_1^{(0)}, \dots, \mathbf{z}_T^{(0)}) \in \mathbb{R}^{T \times d_{\text{word}}}$ được đưa qua bộ mã hóa cấp Từ gồm $L_{\text{word}} = 4$ bước tính toán.

Để tối ưu hóa số lượng tham số mà vẫn đảm bảo chiều sâu mô hình, kiến trúc áp dụng kỹ thuật Cross-layer Parameter Sharing: thay vì khởi tạo 4 khối Transformer Encoder độc lập, mô hình chỉ sử dụng đúng một khối TransformerEncoderLayer vật lý duy nhất ($d_{\text{word}} = 128, H_{\text{word}} = 4, d_{\text{ff}} = 512$) và lặp lại khối này 4 lần:

$$\begin{aligned} \mathbf{Z}^{(l)} = & \text{SharedTransformerBlock}(\mathbf{Z}^{(l-1)}, \text{mask} = \text{attention_mask}), \\ & l = 1, 2, 3, 4 \end{aligned} \quad (3.5)$$

Kỹ thuật này giúp mô hình đạt khả năng trừu tượng hóa ngữ cảnh tương đương mạng 4 lớp nhưng tiết kiệm tới 75% dung lượng tham số của tầng Word Encoder.

3.1.4 Đầu ra đa nhiệm và ràng buộc trọng số (Weight Tying)

Sau L_{word} lớp mã hóa ngữ cảnh, vector đầu ra $\mathbf{z}_i^{(L_{\text{word}})} \in \mathbb{R}^{128}$ tại vị trí token i được phân nhánh vào hai đầu ra chuyên biệt:

- **Detector Head (Bộ phát hiện lỗi nhị phân):** Gồm mạng nơ-ron truyền thẳng (MLP) 2 tầng với hàm kích hoạt GELU và Dropout:

$$\hat{\mathbf{y}}_i^{\text{det}} = \mathbf{W}_{d2} \cdot \text{Dropout} \left(\text{GELU} \left(\mathbf{W}_{d1} \mathbf{z}_i^{(L_{\text{word}})} + \mathbf{b}_{d1} \right) \right) + \mathbf{b}_{d2} \in \mathbb{R}^2 \quad (3.6)$$

với $\mathbf{W}_{d1} \in \mathbb{R}^{64 \times 128}$, $\mathbf{b}_{d1} \in \mathbb{R}^{64}$, $\mathbf{W}_{d2} \in \mathbb{R}^{2 \times 64}$, $\mathbf{b}_{d2} \in \mathbb{R}^2$. Đầu ra biểu diễn phân phối xác suất nhị phân {đúng: 0, lỗi: 1}.

- **Corrector Head (Bộ sửa từ) & Weight Tying:** Vector ngữ cảnh 128 chiều được chiếu về không gian nhúng từ 64 chiều thông qua ma trận $\mathbf{W}_{\text{proj}} \in \mathbb{R}^{64 \times 128}$. Sau đó, thay vì khởi tạo một ma trận phân loại mới kích thước 9.500×64 , mô hình chia sẻ trực tiếp ma trận trọng số với bảng Word Embedding ban đầu (Weight Tying):

$$\hat{\mathbf{y}}_i^{\text{corr}} = \left(\mathbf{W}_{\text{proj}} \mathbf{z}_i^{(L_{\text{word}})} \right) \cdot \mathbf{W}_{\text{word_embeddings}}^T + \mathbf{b}_{\text{corr}} \in \mathbb{R}^{|\mathcal{V}_{\text{word}}|} \quad (3.7)$$

với $\mathbf{W}_{\text{word_embeddings}} \in \mathbb{R}^{9.500 \times 64}$ và $\mathbf{b}_{\text{corr}} \in \mathbb{R}^{9.500}$. Kỹ thuật này giúp mô hình tiết kiệm thêm 608.000 tham số độc lập và đồng bộ không gian biểu diễn ngữ nghĩa giữa đầu vào và đầu ra.

3.1.5 Hàm mất mát tổng hợp

Mô hình được huấn luyện đồng thời theo cơ chế tối ưu đa nhiệm:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{det}} + \mathcal{L}_{\text{corr}} \quad (3.8)$$

- $\mathcal{L}_{\text{det}}$ là hàm mất mát Cross-Entropy nhị phân tính trên toàn bộ các token thực tế trong câu:

$$\mathcal{L}_{\text{det}} = -\frac{1}{N_{\text{valid}}} \sum_{i \in \text{valid}} \log P(\hat{d}_i = d_i^{\text{true}}) \quad (3.9)$$

- $\mathcal{L}_{\text{corr}}$ là hàm mất mát Cross-Entropy đa lớp trên 9.500 từ vựng, chỉ được tính toán tại các vị trí token thực sự có lỗi ($d_i^{\text{true}} = 1$):

$$\mathcal{L}_{\text{corr}} = -\frac{1}{\sum_i \mathbb{I}(d_i^{\text{true}} = 1)} \sum_{i: d_i^{\text{true}} = 1} \log P(\hat{y}_i^{\text{corr}} = y_i^{\text{target}}) \quad (3.10)$$

3.1.6 Bảng phân rã chi tiết tham số cấu hình

Mô hình được thiết kế tối ưu với tổng cộng đúng **1.001.310 tham số có thể huấn luyện**, phân rã chi tiết từng thành phần trong Bảng 3.1:

Thành phần mô hình	Số tham số	Tỷ lệ (%)
Word Embeddings	608.000	60.72%
Character Embeddings	25.600	2.56%
Char Position Embeddings	2.048	0.20%
Word Position Embeddings	24.576	2.45%
Character Encoder (2 lớp)	99.840	9.97%
Feature Fusion Layer	16.768	1.67%
Shared Word Encoder	198.400	19.76%
Detector Head	8.386	0.84%
Corrector Projection & Bias	17.692	1.77%
Tổng số tham số (Trainable)	1.001.310	100.0%

Bảng 3.1: Phân rã chi tiết số lượng tham số của mô hình.

Chương 4

HUẤN LUYỆN, THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

4.1 Môi trường thực nghiệm và kỹ thuật tối ưu hóa

4.1.1 Hạ tầng phần cứng và môi trường phần mềm

Toàn bộ quá trình huấn luyện và đánh giá thực nghiệm của mô hình được triển khai trên nền tảng Kaggle với cấu hình cụ thể như sau:

- **Hạ tầng phần cứng (GPU):** Sử dụng đơn GPU **NVIDIA Tesla T4** với dung lượng bộ nhớ 16 GB VRAM.
- **Môi trường phần mềm:**
 - Hệ điều hành: Linux (Ubuntu trên môi trường Kaggle).
 - Ngôn ngữ & Thư viện cốt lõi: Python 3.11+, PyTorch 2.5 tích hợp sẵn CUDA và cuDNN để tăng tốc phần cứng.
 - Thư viện hỗ trợ: `huggingface_hub` (tự động tải và đồng bộ tập dữ liệu), `torch.amp` (tự động tối ưu Mixed Precision FP16 với `GradScaler`).

4.1.2 Kỹ thuật huấn luyện tối ưu bộ nhớ

Do tập dữ liệu có quy mô hàng triệu câu, việc nạp toàn bộ vào bộ nhớ chính (RAM) sẽ gây lỗi tràn bộ nhớ (Out-Of-Memory). Nhóm áp dụng các giải pháp kỹ thuật tối ưu hóa luồng dữ liệu và tính toán:

- **Streaming DataLoader (`JsonlBucketBatches`):** Đọc và giải mã dữ liệu trực tiếp dạng luồng (stream) từ tệp tin JSONL trên đĩa, kết hợp cơ chế bộ đệm phân vùng (`bucket_size = 4096`) để gom cụm các câu có độ dài tương đồng trước khi nạp vào mini-batch. Kỹ thuật này vừa giải phóng 100% áp lực nạp RAM, vừa triệt tiêu chi phí tính toán các token padding dư thừa trên GPU.

- **Huấn luyện Mixed Precision (BF16 / FP16):** Tận dụng cơ chế tự động ép kiểu chính xác hỗn hợp (`torch.cuda.amp`). Mô hình ưu tiên định dạng Bfloat16 (hoặc Float16 kết hợp `GradScaler` trên các thế hệ GPU cũ), giúp giảm 50% dung lượng VRAM tiêu thụ và tăng tốc độ nhân ma trận của các khối Transformer Encoder.

4.1.3 Thiết lập siêu tham số huấn luyện (Hyperparameters)

Mô hình được huấn luyện trong tổng cộng **10 epochs** với cấu hình siêu tham số được chuẩn hóa như sau:

- **Kích thước Batch (Batch size):** 64 mẫu/batch.
- **Thuật toán tối ưu (Optimizer):** AdamW ($\text{lr} = 10^{-4}$, $\beta = (0.9, 0.95)$, $\text{weight_decay} = 0.01$).
- **Chiến lược điều chỉnh tốc độ học (LR Schedule):** Linear Warmup trong 2 epoch đầu tiên nhằm ổn định các trọng số nhúng ngẫu nhiên, sau đó duy trì tốc độ học cố định xuyên suốt các epoch còn lại.
- **Kích thước không gian biểu diễn:** Không gian từ vựng $|V_{\text{word}}| = 9.500$, không gian ký tự $|V_{\text{char}}| = 400$, số chiều nhúng ký tự $d_{\text{char}} = 64$, số chiều ẩn từ $d_{\text{word}} = 128$, tỷ lệ Dropout $p = 0.1$.
- **Độ dài chuỗi tối đa:** Giới hạn 192 token/câu và 32 ký tự/token.

4.2 Đánh giá kết quả và hiệu năng mô hình

Sau toàn bộ tiến trình huấn luyện 10 epoch, trọng số tối ưu nhất trên tập kiểm định được tự động trích xuất và lưu trữ thành tệp checkpoint (**best.pt**). Các chỉ số đánh giá được phân tích trên hai nhiệm vụ cốt lõi: Phát hiện lỗi (Detection) và Sửa lỗi (Correction).

4.2.1 Mức độ hội tụ của hàm mất mát

Giá trị mất mát tổng thể trên tập kiểm định (**Val Loss**) đạt mốc tối ưu **1,5243**. Do hàm mục tiêu $\mathcal{L}_{\text{total}}$ là sự kết hợp đồng thời giữa hàm mất mát phân loại nhị phân (phát hiện lỗi) và hàm mất mát Cross-Entropy trên không gian 9.500 từ vựng (sửa từ), giá trị mất mát 1,52 phản ánh rằng mô hình đã thu hẹp không gian phân phối xác suất từ 9.500 từ ứng viên xuống chỉ còn trung bình khoảng $\exp(1,25) \approx 3,5$ từ tiềm năng nhất cho mỗi vị trí token bị sai chính tả.

4.2.2 Hiệu năng phát hiện lỗi

Bộ phát hiện lỗi nhị phân (Detector Head) thể hiện độ ổn định và độ tin cậy vượt trội:

- **Độ chính xác phát hiện (Precision):** Đạt **92,43%**. Chỉ số Precision cao có ý nghĩa then chốt trong các bài toán sửa lỗi chính tả thực tế, đảm bảo hệ thống cực kỳ hạn chế việc gắn cờ sai hoặc sửa nhầm các từ đúng trong câu của người dùng.
- **Độ phủ phát hiện (Recall):** Đạt **83,27%**, cho thấy mô hình bắt được phần lớn các lỗi sai âm vị, ký tự và chính tả phổ biến trong tiếng Việt.
- **F1-Score tổng thể:** Đạt **87,61%**, khẳng định sự cân bằng tối ưu giữa việc phát hiện triệt để lỗi và bảo toàn tính toàn vẹn của văn bản gốc.

4.2.3 Hiệu năng sửa lỗi toàn diện (End-to-End Correction)

Hiệu năng khôi phục từ đúng của nhánh sửa lỗi (Corrector Head) và toàn bộ pipeline xử lý đạt kết quả khả quan:

- **Độ chính xác thay thế từ (Corrector Accuracy):** Đạt **77,07%**. Khi vị trí lỗi đã được phát hiện chính xác, hơn 3/4 trường hợp mô hình đã dự đoán đúng chính xác từ gốc ban đầu nhờ cơ chế ràng buộc ngữ cảnh hai chiều của Word Encoder và liên kết trọng số (Weight Tying).
- **Độ phủ sửa lỗi đầu-cuối (End-to-End Correction Recall):** Đạt **74,95%**. Điều này đồng nghĩa với việc xét trên toàn bộ tập lỗi xuất hiện trong dữ liệu thực nghiệm, mô hình có khả năng vừa định vị đúng vị trí lỗi, vừa khôi phục hoàn chỉnh về đúng từ vựng chuẩn xác với tỷ lệ 3/4 tổng số lỗi, đồng thời giữ tốc độ suy luận theo thời gian thực nhờ kích thước mô hình siêu gọn ($\sim 1\text{M}$ tham số).

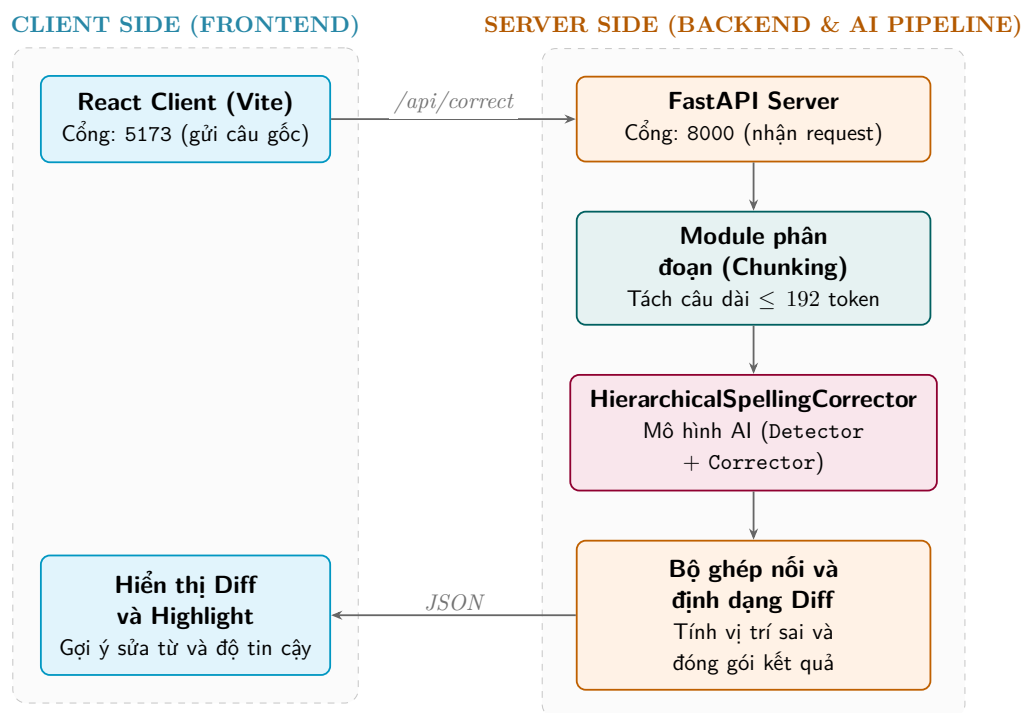
Chương 5

TRIỂN KHAI ỨNG DỤNG VÀ THỰC NGHIỆM WEB DEMO

5.1 Kiến trúc hệ thống Web tương tác

Để đưa mô hình vào ứng dụng thực tiễn, nhóm phát triển một hệ thống Web hoàn chỉnh:

- **Backend Service (FastAPI):** Đóng vai trò cung cấp RESTful API, nạp trực tiếp mô hình từ checkpoint `best.pt` và quản lý suy luận.
- **Frontend Application (React + TypeScript + Vite):** Cung cấp giao diện người dùng hiện đại, hỗ trợ gõ trực tiếp, hiển thị nhãn tô màu trực quan, gợi ý sửa lỗi và tải tệp tin văn bản `.txt`.



Hình 5.1: Kiến trúc luồng xử lý và triển khai của ứng dụng Web Demo.

5.2 Các tính năng kỹ thuật nổi bật

- **Cơ chế phân đoạn chuỗi dài (Text Chunking):** Đối với các văn bản dài (tối đa 5.000 ký tự), backend tự động chia nhỏ thành các đoạn không quá 192 token theo ranh giới câu, đồng thời bảo toàn nguyên vẹn định dạng thụt lề, khoảng trắng và xuống dòng ban đầu.
- **Ba chế độ điều chỉnh độ nhạy (Sensitivity Modes):**
 - **Conservative (Ngưỡng 0.80):** Chế độ thận trọng, chỉ can thiệp sửa khi mô hình có độ tự tin cực cao.
 - **Balanced (Ngưỡng 0.50):** Chế độ cân bằng tiêu chuẩn, phù hợp với hầu hết các văn bản thông thường.
 - **Aggressive (Ngưỡng 0.30):** Chế độ nhạy cao, tối đa hóa việc bắt các lỗi chính tả tinh vi và teencode.
- **Trực quan hóa sai khác (Diff Visualizer):** Giao diện hiển thị trực quan các từ bị sửa đổi bằng màu sắc nổi bật, cho phép người dùng xem từ gốc, từ gợi ý và sao chép nhanh kết quả chỉ với một thao tác.

Chương 6

KẾT LUẬN VÀ HƯỞNG PHÁT TRIỂN

6.1 Kết luận

Dự án đã nghiên cứu và triển khai thành công mô hình **Hierarchical Transformer Encoder** gọn nhẹ (1M tham số) giải quyết bài toán sửa lỗi chính tả tiếng Việt. Các đóng góp chính của đề tài bao gồm:

1. Xây dựng pipeline xử lý dữ liệu chuẩn chỉnh với 4,66 triệu mẫu câu tổng hợp, tích hợp đầy đủ 10 dạng lỗi chính tả tiếng Việt thực tế.
2. Áp dụng thành công các kỹ thuật tối ưu hóa tham số (Masked Mean Pooling, ALBERT Weight Sharing, Weight Tying) giúp mô hình đạt hiệu năng cao với chi phí tài nguyên tối thiểu.
3. Đạt kết quả thực nghiệm ấn tượng trên tập kiểm định với **F1 phát hiện lỗi 87.73%** và **End-to-End Recall 74.95%**.
4. Đóng gói hoàn chỉnh hệ sinh thái từ huấn luyện phân tán đến ứng dụng Web tương tác người dùng.

6.2 Hạn chế của mô hình

Mặc dù đạt được hiệu quả cao về độ chính xác và tốc độ suy luận, mô hình vẫn tồn tại một số hạn chế nhất định cần được cải thiện:

- **Cơ chế phân loại 1-1:** Do kiến trúc Corrector hiện tại hoạt động theo cơ chế phân loại token song song độc lập, mô hình chưa thể giải quyết các dạng lỗi phức tạp làm thay đổi số lượng từ như lỗi dính từ (ví dụ: `đi học` → `đi học`) hoặc lỗi tách từ (ví dụ: `b ảo` → `bảo`), cũng như lỗi thừa/thiếu từ.
- **Vấn đề ngoài từ điển (Out-Of-Vocabulary):** Không gian từ vựng cố định $|V_{\text{word}}| = 9.500$ từ vựng giúp tối ưu hóa tốc độ và bộ nhớ, bao quát

hầu hết các âm tiết tiếng Việt chuẩn. Tuy nhiên các từ mục tiêu nằm ngoài từ điển sẽ không thể khôi phục chính xác bằng bộ phân loại này.

- **Ngữ cảnh liên câu bị giới hạn theo phân đoạn (Chunking Boundary):** Giới hạn độ dài ngữ cảnh 192 token đòi hỏi văn bản dài phải được chia thành các chunk độc lập, có thể làm giảm bớt tính liên kết ngữ nghĩa giữa các câu nằm ở ranh giới phân tách.

6.3 Hướng phát triển trong tương lai

- **Mở rộng kiến trúc xử lý lỗi dính/tách từ:** Nghiên cứu kết hợp cơ chế giải mã tự hồi quy (Autoregressive Decoder / Seq2Seq) hoặc Tokenizer cấp Subword (BPE/Byte-level) để khắc phục triệt để hạn chế của cơ chế phân loại 1-to-1.
- **Mở rộng từ điển từ đồng âm ngữ cảnh:** Tích hợp thêm các cặp từ đồng âm dễ nhầm lẫn như *chẩn đoán/chuẩn đoán*, *sơ suất/sơ xuất*, *xuất sắc/suất sắc*.
- **Mở rộng API và Tiện ích mở rộng:** Đóng gói mô hình thành tiện ích kiểm tra chính tả trực tiếp trên trình duyệt Web (Chrome/Firefox extension).

PHÂN CÔNG CÔNG VIỆC

Để hoàn thành dự án một cách tốt nhất, công việc được phân chia công bằng và phù hợp với vai trò của từng thành viên trong nhóm như sau:

Thành viên	Nhiệm vụ cụ thể
Nguyễn Đoàn Nhật Minh	- Tìm hiểu cơ sở lý thuyết về bài toán sửa lỗi chính tả tiếng Việt và kiến trúc Transformer Encoder phân cấp. - Khảo sát, phân loại các dạng lỗi chính tả tiếng Việt và xây dựng nội dung tổng quan của báo cáo. - Tham gia kiểm thử mô hình trên các nhóm lỗi thực tế, tổng hợp nhận xét và hoàn thiện báo cáo.
Nguyễn Ngọc Bình An	- Thiết kế và cài đặt mô hình Hierarchical Transformer Encoder 1M tham số. - Thiết kế và xây dựng pipeline trích xuất văn bản từ Wikipedia và sinh nhiễu với 10 loại lỗi. - Hỗ trợ xây dựng pipeline sinh dữ liệu nhiễu tổng hợp, kiểm tra chất lượng dữ liệu và phòng tránh rò rỉ tập kiểm thử.
Phạm Lê Việt Đức	- Nghiên cứu quy trình tiền xử lý dữ liệu, xây dựng tập từ vựng và chuẩn hóa dữ liệu đầu vào. - Phát triển Backend API bằng FastAPI và ứng dụng Frontend bằng React/TypeScript. - Tích hợp hệ thống, kiểm thử quy trình suy luận và đóng gói ứng dụng Web Demo.
Nguyễn Đình Phú	- Xây dựng pipeline huấn luyện streaming tối ưu bộ nhớ và Mixed Precision BF16. - Huấn luyện mô hình, tinh chỉnh siêu tham số, phân tích kết quả qua các epoch và trình bày kiến trúc kỹ thuật. - Thực hiện đánh giá mô hình, tổng hợp các chỉ số thực nghiệm và xây dựng bảng, biểu phục vụ báo cáo.

Bảng 6.1: Bảng phân công nhiệm vụ và vai trò thành viên trong đề tài.